PS팀 AI 역량강화:보안 - Java 언어로 구성된 프로젝트의 소스코드 정적 분석 도구 개발 - 사용자 매뉴얼

PS팀 AI 역량강화:보안 - Java 언어로 구성된 프로젝트의 소스코드 정적 분석 도구 개발 - 사용자 매뉴얼

> **행정안전부 소프트웨어 보안약점 진단가이드 (2021) 기반**
> Java 소스코드 정적 분석 도구 · 핵심 사용자 매뉴얼

목차

1. 설치 및 환경 설정
2. 사용 방법
3. 결과 확인
4. 자주 묻는 질문 (FAQ)

---

1. 설치 및 환경 설정

1.1 사전 요구사항

항목	최소 버전	확인 명령
JDK	17 이상	`java -version`
Maven	3.8 이상	`mvn -version`
메모리	512MB 이상 권장	—

> **참고:** JDK 21을 사용해도 동작합니다. Spring Boot 3.x는 JDK 17+를 요구합니다.

1.2 JDK 17 설치 (Amazon Linux 2023 / EC2 기준)

```
# Corretto 17 설치
sudo dnf install -y java-17-amazon-corretto-devel

# 설치 확인
java -version
# openjdk version "17.x.x" ...
```

1.3 Maven 설치

```
# Maven 설치
sudo dnf install -y maven

# 설치 확인
mvn -version
# Apache Maven 3.x.x
```

1.4 소스 빌드

```
# 프로젝트 디렉터리로 이동
cd java-sast-tool

# 전체 빌드 (테스트 포함)
mvn clean package

# 빠른 빌드 (테스트 생략)
mvn clean package -DskipTests

# 빌드 결과물 확인
ls target/sast.jar
```

빌드 성공 시 `target/sast.jar` (Fat JAR)가 생성됩니다.

1.5 EC2에서 웹 서버로 실행

```
# 웹 서버 시작 (포트 5000, 백그라운드 실행)
nohup java -jar target/sast.jar > sast.log 2>&1 &

# 시작 확인
tail -f sast.log
# ... Started SastApplication in X.XXX seconds
```

브라우저에서 `http://teskim-vibecoding.ddns.net:5000` 에 접속합니다.

> **주의:** EC2 보안 그룹에서 TCP 포트 5000 인바운드 규칙을 허용해야 합니다.

---

2. 사용 방법

2.1 웹 UI 모드 (권장)

1. 브라우저에서 `http://teskim-vibecoding.ddns.net:5000` 접속
2. 분석할 Java 프로젝트를 **ZIP 또는 7z**으로 압축
3. 업로드 영역에 파일을 드래그하거나 클릭하여 선택
4. **"분석 시작"** 버튼 클릭
5. 분석 완료 후 결과 대시보드로 자동 이동

> 최대 업로드 파일 크기: **100MB** · `.zip`, `.7z` 형식 허용

2.2 CLI 모드 (서버리스 / 자동화)

웹 서버 없이 명령줄에서 직접 실행합니다.

```
# 기본 실행: <소스 디렉터리> <리포트 출력 경로>
java -cp target/sast.jar com.sast.SASTEngine <소스_디렉터리> <리포트_파
        일.md>

# 예시: 현재 프로젝트 분석
java -cp target/sast.jar com.sast.SASTEngine ./src/main/java ./
        report.md

# 예시: 특정 프로젝트 분석
java -cp target/sast.jar com.sast.SASTEngine /home/ec2-user/
        myproject/src ./output/report.md
```

CLI 실행 시 다음 파일이 자동 생성됩니다.

파일	내용
`report.md`	Markdown 형식 상세 리포트
`report.json`	JSON 형식 (CI/CD 연동용)
`report.pdf`	PDF 형식 (한국어)

2.3 Maven 명령어 정리

```
# 전체 빌드 + 테스트
mvn clean package

# 테스트만 실행
mvn test

# 특정 테스트 클래스만 실행
mvn test -Dtest=SanitizerFalsePositiveTest

# 코드 커버리지 리포트 생성
mvn test jacoco:report
```

```
# Fat JAR 위치
target/sast.jar
```

2.4 분석 대상 압축 방법

```
# Maven 프로젝트 압축 예시 (src 디렉터리만)
cd /path/to/your-project
zip -r project-src.zip src/

# 프로젝트 전체 압축 (.git, target 제외)
zip -r project.zip . --exclude "*.git*" --exclude "*/target/*"

# 7z 형식으로 압축
7z a project.7z src/
```

---

3. 결과 확인

3.1 웹 대시보드 구성

분석 완료 후 표시되는 대시보드는 다음과 같이 구성됩니다.

```
┌──────────────────────────────────────────────────────────────┐
│  Java SAST Engine              [PDF 다운로드] [새 분석]      │
├──────────────────────────────────────────────────────────────┤
│  project.zip — 분석 파일 수: 12개 · 총 탐지: 8건             │
│                                                              │
│  [  CRITICAL: 1]  [  HIGH: 3]  [  MEDIUM: 2]  [○ LOW: 2] │
├──────────────────────────────────────────────────────────────┤
│  ▼ [HIGH] IV-1.1 — SQL 삽입     BoardController.java│
│    Line 42 · CWE-89                          [상세 보기 →]   │
│     ┌──────────────────────────────────────────────────┐    │
│     │ String sql = "SELECT * FROM user WHERE id='" + id;  │  │
│     └──────────────────────────────────────────────────┘    │
│    오염 흐름: [Source L3: getParameter("id")] → [sql] →      │
│              [Sink L42: executeQuery(sql)]                   │
├──────────────────────────────────────────────────────────────┤
│  ▼ [HIGH] IV-1.3 — 경로 조작     FileController.java │
│  ...                                                         │
└──────────────────────────────────────────────────────────────┘
```

3.2 상세 모달 (Bad/Good Code 비교)

각 탐지 항목에서 **"상세 보기"** 버튼 클릭 시 모달 창이 열립니다.

```
┌──────────────────────────── 상세 정보 ────────────────────────────┐
│  IV-1.1 — SQL 삽입                                   [CWE-89]     │
│  PART4 제1절 1항 (p.180-193)                                      │
```

```
├──────────────────────────────────────────────────────────────────┤
│  탐지 근거                                                        │
│  외부 입력값 'id'(request.getParameter("id"), L3)이              │
│  검증 없이 stmt.executeQuery(sql)(L42)에 사용됩니다.             │
├──────────────────────────────────────────────────────────────────┤
│  권고 수정 가이드                                                 │
│  [ 나쁜 예시]   [ 좋은 예시]                                      │
│                                                                   │
│  나쁜 예시                                                        │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ String sql = "SELECT * FROM user WHERE id='" + id;       │    │
│  │ Statement stmt = con.createStatement();                  │    │
│  │ ResultSet rs = stmt.executeQuery(sql);                   │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                   │
│  좋은 예시                                                        │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ String sql = "SELECT * FROM user WHERE id = ?";          │    │
│  │ PreparedStatement ps = con.prepareStatement(sql);        │    │
│  │ ps.setString(1, id);                                     │    │
│  │ ResultSet rs = ps.executeQuery();                        │    │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

3.3 위험도 판정 기준

위험도	아이콘	판정 기준
CRITICAL		원격 코드 실행, 시스템 완전 장악 가능
HIGH		인증 우회, 데이터 탈취, 경로 탈출 가능
MEDIUM		제한적 정보 노출, 서비스 장애 가능
LOW	○	잠재적 위험, 직접 악용 어려움

> 테스트 코드(`src/test/` 경로) 내 탐지 항목은 자동으로 **LOW**로 하향 조정됩니다.

3.4 오염 흐름(Taint Flow) 해석

```
[Source L3: request.getParameter("id")] → [Propagators: id, sql] →
[Sink L42: stmt.executeQuery(sql)]
```

요소	의미
Source	외부 입력이 코드로 진입하는 지점 (라인 번호 포함)
Propagators	오염값이 거쳐 간 변수 이름 목록
Sink	오염값이 위험 API에 도달한 지점 (라인 번호 포함)

3.5 CLI 리포트 (Markdown) 예시

```
### [1] IV-1.1 — SQL 삽입

| 항목 | 내용 |
|------|------|
| **위험도** | 높음(HIGH) |
| **파일** | `com/example/BoardController.java` |
| **라인** | 42 |
| **CWE** | CWE-89 |

#### 오염 흐름
[Source L3: getParameter("id")] → [Propagators: [id, sql]] →
        [Sink L42: executeQuery(sql)]

#### 취약한 코드
...

#### 권고 수정 코드
...
```

---

4. 자주 묻는 질문 (FAQ)

Q. ZIP 업로드 후 "분석 중 오류가 발생했습니다"가 뜹니다.

A. 파일 크기가 100MB를 초과하거나, `.java` 파일이 없는 ZIP일 수 있습니다. ZIP 내부에 Java 소스 파일이 포함되어 있는지 확인하세요.

Q. 탐지 결과가 너무 많이 나옵니다 (오탐 의심).

A. `sast-suppressions.json`에 억제 규칙을 추가하거나, 코드에 `@SuppressWarnings("sast-ignore")`를 추가하세요. 또한 Sanitizer 메서드를 사용하고 있다면 `security-rules.json`의 `sanitizers` 배열에 해당 메서드명을 추가하면 오탐이 줄어듭니다.

Q. PDF가 생성되지 않거나 한국어가 깨집니다.

A. PDF 생성은 OpenPDF 라이브러리를 사용합니다. 서버에 한국어 폰트(나눔고딕)가 없는 경우 폴백 폰트로 대체될 수 있습니다. `sast.log`에서 PDF 관련 경고를 확인하세요.

Q. CI/CD 파이프라인에 통합하려면 어떻게 하나요?

A. CLI 모드를 사용하고, 생성된 `report.json`을 파싱하여 CRITICAL/HIGH 건수가 0 이상이면 빌드를 실패 처리하는 스크립트를 추가하세요.

```bash
# CI/CD 통합 예시 (Bash)
java -cp target/sast.jar com.sast.SASTEngine ./src ./report.md
CRITICAL=$(jq '[.findings[] | select(.severity=="CRITICAL")] |
        length' report.json)
if [ "$CRITICAL" -gt 0 ]; then
  echo "CRITICAL 취약점 ${CRITICAL}건 탐지 — 빌드 실패"
  exit 1
fi
```

Q. 규칙을 추가했는데 탐지가 안 됩니다.

A. `taintAnalysis: true`인 규칙은 `TaintAnalysisEngine.RULE_SINKS`에 해당 `ruleId`와 Sink 메서드 목록을 등록해야 합니다. 등록하지 않으면 Sink 탐지가 동작하지 않습니다. 재빌드(`mvn clean package`) 후 테스트하세요.

Java SAST Engine v1.0.0 — 행정안전부 소프트웨어 보안약점 진단가이드 2021 기반